

FACULTAD DE INFORMÁTICA

Máster en Internet de las Cosas
2025-2026



Asignatura:

Diseño de Infraestructura Inteligente para IOT (DII)

Práctica:

Trabajos individuales grupales

Alumno:

Ariel Gámez Díaz

Sumario

Estado del Arte.....	3
Entornos Blockchain Determinísticos con Docker y Anvil.....	4
2.0 Ejemplo 1: Interacción Básica de Billetera y Transferencias.....	5
2.1 Objetivo y Componentes Clave.....	5
2.2 Análisis del Flujo de Ejecución.....	6
3.0 Ejemplo 2: Despliegue e Interacción con un Smart Contract (PiggyBank).....	7
3.1 Arquitectura del Ejemplo.....	7
3.2 Análisis del Flujo de Ejecución Automatizado.....	8
3.3 Evaluación de los Casos de Uso.....	9
4.0 Ejemplo 3: Ejemplo 3: Integración Avanzada de IoT y Blockchain.....	9
4.1 Arquitectura del Sistema End-to-End.....	10
4.2 Desglose de Componentes y sus Funciones.....	10
4.2.1 El Dispositivo IoT (ESP32).....	10
4.2.2 El Broker de Mensajería (Mosquitto).....	10
4.2.3 El Oráculo Python (bridge.py).....	11
4.2.4 La Blockchain y el Smart Contract (SensorData.sol).....	11
4.3 Análisis del Flujo de Datos Completo.....	11
Valor de la Arquitectura.....	12
Conclusión: Síntesis de Casos de Uso.....	13



Estado del Arte

En el desarrollo de aplicaciones blockchain, la capacidad de reproducir resultados de manera consistente es un pilar fundamental para la calidad y la fiabilidad. Los entornos de desarrollo determinísticos son ecosistemas controlados y aislados donde cada operación, desde la creación de una cuenta hasta la ejecución de un contrato complejo, produce siempre el mismo resultado. Esta previsibilidad es de una importancia estratégica incalculable, ya que elimina la variabilidad inherente de las redes de prueba públicas, cuyos estados cambian constantemente. La reproducibilidad se convierte así en un requisito indispensable para la automatización de pruebas, la integración en pipelines de CI/CD, la ejecución de talleres técnicos y la realización de demostraciones impecables.

Para construir estos entornos robustos, esta guía se apoya en una combinación de herramientas líderes en la industria. **Docker** proporciona la capacidad de empaquetar y aislar aplicaciones con todas sus dependencias en contenedores portátiles. **Anvil**, una herramienta del framework **Foundry**, nos permite ejecutar una instancia local de la Ethereum Virtual Machine (EVM) de forma instantánea y predecible. Al combinar Docker para la contenerización y Anvil para la simulación de la blockchain, podemos orquestar flujos de trabajo complejos y garantizar que se comporten de manera idéntica en cualquier máquina y en cualquier momento.

A lo largo de esta guía, exploraremos tres ejemplos prácticos de complejidad creciente que demuestran el poder de esta arquitectura, comenzando con las interacciones más básicas para sentar una base sólida sobre la cual construir sistemas más avanzados.

Entornos Blockchain Determinísticos con Docker y Anvil

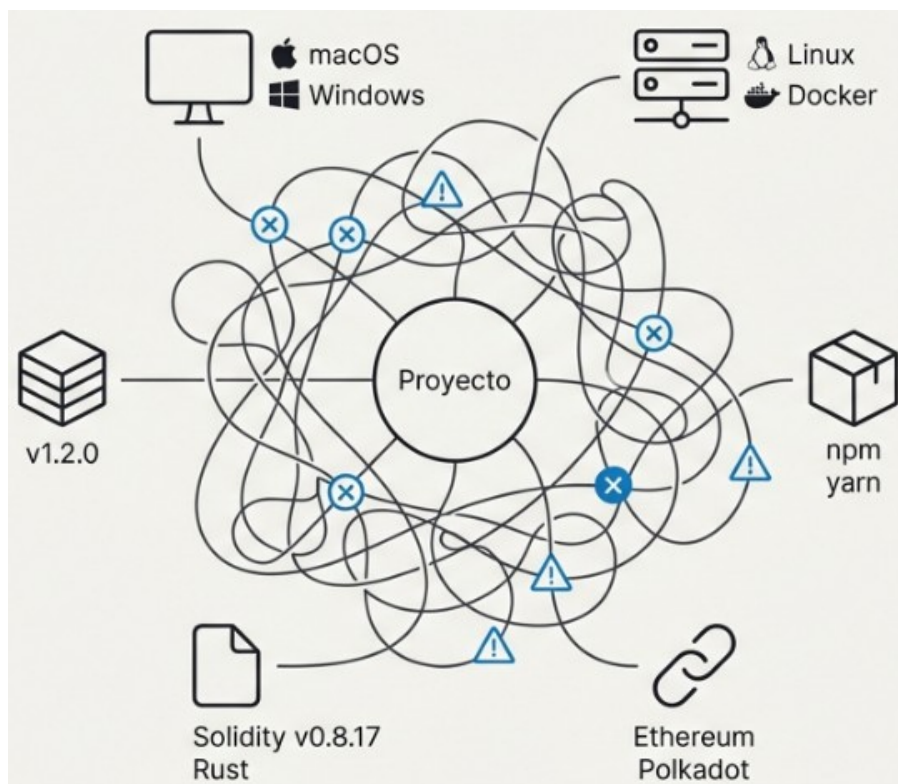


Figura 1: El caos de los entornos de desarrollo blockchain



Figura 2: La solución aislamiento y potencia

2.0 Ejemplo 1: Interacción Básica de Billetera y Transferencias

Este primer ejemplo establece los cimientos de nuestra metodología al demostrar cómo automatizar las interacciones más fundamentales de una blockchain: la creación de billeteras y la transferencia de fondos. Al encapsular una red Anvil dentro de un contenedor Docker y controlarla desde el exterior mediante scripts, logramos un entorno de prueba simple pero potente, ideal para validar la lógica básica de las transacciones y familiarizarse con la gestión de cuentas en un entorno controlado.

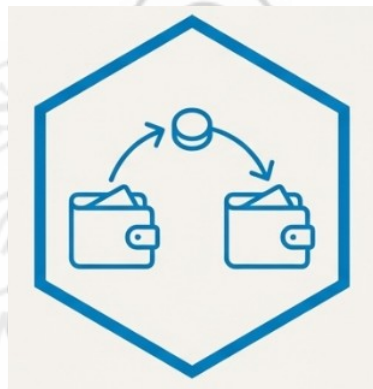


Figura 3: NIVEL 1: EL SANDBOX ATÓMICO

2.1 Objetivo y Componentes Clave

El propósito de este montaje es automatizar un escenario simple pero esencial: crear mediante un script una nueva billetera para un usuario ficticio, "Bob", transferirle fondos desde una cuenta prefinanciada, "Alice", y verificar que los saldos se actualicen correctamente.

Los componentes que intervienen en este flujo son:

- **Dockerfile:** Define el entorno de ejecución, utilizando la imagen base específica `ghcr.io/foundry-rs/foundry:stable` para garantizar que todas las herramientas necesarias estén disponibles y versionadas.
- **flow.sh:** Un script Bash que actúa como orquestador. Contiene la secuencia de comandos para interactuar con la blockchain desde la máquina anfitriona (host).
- **Contenedor Anvil (red - local):** La instancia de la blockchain en sí, ejecutándose de forma aislada dentro de Docker y exponiendo un punto de acceso para las transacciones.

2.2 Análisis del Flujo de Ejecución

A El script `flow.sh`, ejecutándose en la máquina host, actúa como un orquestador que dirige las operaciones dentro del contenedor. Utiliza comandos `docker exec` para enviar instrucciones directamente al contenedor `red-local`, demostrando un patrón clave de interacción `host-contenedor` para la automatización.

El proceso se desarrolla en los siguientes pasos:

1. **Creación de la billetera de Bob:** El script invoca un comando dentro del contenedor para generar una nueva cuenta, simulando la incorporación de un nuevo usuario al sistema.
2. **Verificación del saldo inicial:** Se consulta el saldo de la cuenta recién creada de Bob, confirmando que es de cero, como se esperaba.
3. **Envío de fondos desde Alice:** El script ejecuta una transacción para enviar 5 ETH desde una de las cuentas prefinanciadas por defecto en Anvil (que representa a "Alice") a la nueva dirección de Bob.
4. **Verificación del saldo final:** Finalmente, se vuelve a consultar el saldo de Bob para confirmar que ha recibido los 5 ETH, validando así que la transferencia se ha completado con éxito.



Figura 4: Automatizando una Transferencia Simple

Este patrón, aunque sencillo, es invaluable. Sirve como un bloque de construcción fundamental para pruebas de scripts, configuración inicial de entornos para demos (donde se necesita pre-financiar cuentas) y como un excelente punto de partida para que los desarrolladores se familiaricen con la gestión programática de cuentas y saldos en Anvil sin la complejidad de los contratos inteligentes.

Con esta base establecida, el siguiente ejemplo eleva el nivel de complejidad al introducir un contrato inteligente en el flujo de trabajo, demostrando un ciclo de vida de desarrollo más completo.

3.0 Ejemplo 2: Despliegue e Interacción con un Smart Contract (PiggyBank)

Este segundo ejemplo representa un salto cualitativo, ya que demuestra un flujo de trabajo de desarrollo completo y autocontenido. Aquí, no solo gestionamos cuentas y saldos, sino que automatizamos todo el ciclo de vida de un contrato inteligente: desde la inicialización de un proyecto Foundry hasta su compilación, despliegue e interacción, todo orquestado por un único script. Este enfoque encapsula la lógica de la aplicación y sus pruebas en un paquete reproducible, ideal para escenarios profesionales que exigen consistencia y facilidad de uso.

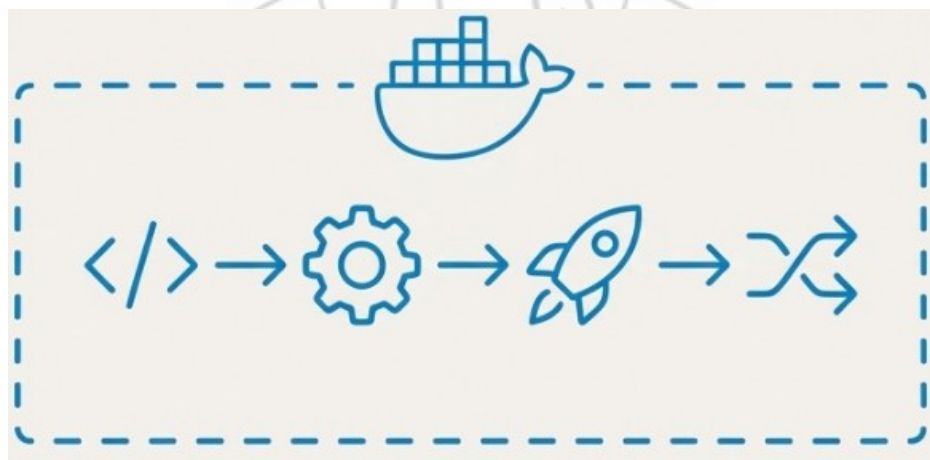


Figura 5: NIVEL 2: LA FÁBRICA DE CONTRATOS

3.1 Arquitectura del Ejemplo

El objetivo principal es demostrar un flujo 100% reproducible para compilar, desplegar e interactuar con el contrato `PiggyBank.sol`. Este escenario simula un ciclo de desarrollo y prueba completo, garantizando que cualquier desarrollador pueda ejecutarlo con un solo comando y obtener exactamente el mismo resultado.

Los componentes clave de esta arquitectura son:

- **Contrato:** `PiggyBank.sol`, un contrato inteligente de ejemplo que sirve como el componente on-chain de nuestra aplicación.
- **Script de Interacción:** `Interact.s.sol`, un script de Foundry que define la secuencia de interacciones con el contrato (despliegue, depósitos, retiros) utilizando las cuentas predeterminadas de Anvil.
- **Orquestador:** `setup_and_run.sh`, el script Bash que actúa como punto de entrada único, automatizando cada paso del proceso, desde la creación del proyecto hasta la ejecución final.
- **Entorno:** `Dockerfile`, que prepara un contenedor basado en `ghcr.io/foundry-rs/foundry:latest` con todas las dependencias necesarias para ejecutar el flujo de manera aislada.

3.2 Análisis del Flujo de Ejecución Automatizado

El script `setup_and_run.sh` es el corazón de este ejemplo, gestionando todo el ciclo de vida desde cero. Al ejecutarse, realiza una serie de acciones secuenciales que emulan el trabajo de un desarrollador:

1. **Inicialización del proyecto:** Ejecuta `forge init` para crear una nueva estructura de proyecto de Foundry.
2. **Instalación de dependencias:** Descarga las librerías necesarias, como `forge-std`, para asegurar que el proyecto tenga todas sus dependencias resueltas.
3. **Configuración del proyecto:** Configura el archivo `foundry.toml` con los remappings correctos, estableciendo una configuración conocida y válida para la compilación.
4. **Compilación de los contratos:** Compila `PiggyBank.sol` y el script de interacción para generar el bytecode necesario.
5. **Ejecución del script de interacción:** Lanza el script `Interact.s.sol`, que se conecta a la red Anvil en ejecución, despliega el contrato e interactúa con él utilizando varias cuentas prefinanciadas.



Figura 6: Un contrato, un script y cero sorpresas :)

3.3 Evaluación de los Casos de Uso

Esta arquitectura autocontenida es extremadamente valiosa para escenarios profesionales que requieren fiabilidad y simplicidad de ejecución. Su valor se manifiesta en varios contextos:

- **Demos técnicas y Workshops:** Permite a los participantes o clientes ejecutar un escenario complejo con un solo comando, eliminando la fricción de la configuración manual y asegurando una experiencia fluida.
- **CI/CD Local:** Facilita la creación de pipelines de prueba locales que pueden verificar el ciclo de vida completo de un contrato (compilación, despliegue, interacción) antes de integrarlo en el repositorio principal.
- **Pruebas Determinísticas:** Al ejecutarse contra una instancia limpia de Anvil en cada ejecución, garantiza que las pruebas de interacción con el contrato produzcan siempre el mismo resultado, eliminando falsos negativos causados por un estado de red contaminado.

Tras dominar la automatización de contratos, el siguiente ejemplo eleva aún más la complejidad al integrar componentes del mundo real, demostrando cómo conectar dispositivos IoT y servicios de mensajería con nuestra blockchain determinista.

4.0 Ejemplo 3: Ejemplo 3: Integración Avanzada de IoT y Blockchain

Este ejemplo representa la culminación de la guía, simulando un sistema industrial del mundo real para demostrar cómo conectar de manera fiable el mundo físico (off-chain) con un registro digital inmutable (on-chain). La arquitectura se basa en un patrón de "contenedor por servicio", una práctica recomendada en DevOps donde los componentes están desacoplados y se comunican a través de un bus de mensajería. El propósito es mostrar un flujo de datos completo, desde un sensor físico hasta su registro inmutable en una blockchain, todo dentro de un entorno local, controlado y determinista.



Figura 7: NIVEL 3: EL PUENTE AL MUNDO REAL

4.1 Arquitectura del Sistema End-to-End

El objetivo general es crear un puente de confianza donde un dispositivo microcontrolador ESP32 publica datos de sensores, y estos datos son registrados de forma segura e inmutable en nuestra blockchain local gestionada por Anvil. Este sistema replica la arquitectura de una solución industrial, pero con la ventaja de poder probarla y desarrollarla localmente.

El flujo de datos sigue una arquitectura de alto nivel bien definida:

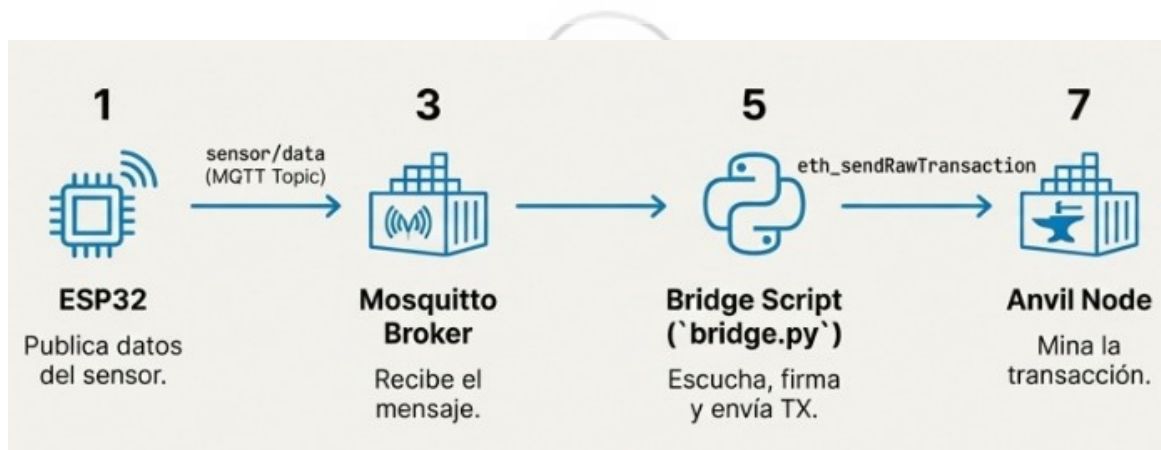


Figura 8: Arquitectura Completa: IoT + Blockchain

4.2 Desglose de Componentes y sus Funciones

Cada componente de la arquitectura cumple un rol específico y se gestiona de forma independiente a través de su propio `Dockerfile` o script de ejecución, lo que demuestra una separación de preocupaciones que es fundamental para sistemas robustos y escalables.

4.2.1 El Dispositivo IoT (ESP32)

Actúa como la fuente de datos del mundo real. Este microcontrolador está programado para publicar periódicamente una carga útil en formato JSON en un tema MQTT específico: `sensor/data`. Es gestionado por su propio script de compilación y ejecución (`esp32_run.sh`), representando un componente de hardware externo al ecosistema Docker principal.

4.2.2 El Broker de Mensajería (Mosquitto)

Funciona como el intermediario de comunicación central. Se ejecuta en un contenedor Docker dedicado, definido por `Dockerfile.mqtt`, que instala y configura el broker `eclipse-mosquitto`. Recibe los mensajes publicados por el ESP32 y los pone a disposición de cualquier servicio que se suscriba al tema correspondiente.

4.2.3 El Oráculo Python (`bridge.py`)

Este script de Python es el componente más crítico, ya que actúa como el "puente" entre el mundo off-chain (MQTT) y el mundo on-chain (Blockchain). Es iniciado y gestionado por el script `bridge.sh`. Su función es:

1. Suscribirse al tema MQTT `sensor/data`.
2. Al recibir un mensaje JSON, lo procesa.
3. Utiliza una clave privada de una cuenta de Anvil para firmar una transacción que contiene los datos del sensor.
4. Envía esta transacción al contrato inteligente desplegado en la red Anvil.

4.2.4 La Blockchain y el Smart Contract (`SensorData.sol`)

La blockchain local se ejecuta en un contenedor gestionado por `Dockerfile.anvil`, que proporciona el entorno Anvil determinista. Dentro de esta red se despliega el contrato `SensorData.sol` utilizando el script de Foundry `DeploySensor.s.sol`. El contrato sirve como un registro inmutable, con una función para almacenar los datos enviados por el oráculo, garantizando su integridad y trazabilidad.

4.3 Análisis del Flujo de Datos Completo

El ciclo completo, desde la generación del dato hasta su confirmación, sigue una secuencia precisa y verificable:

1. El ESP32 se activa y publica un mensaje JSON en el tema MQTT `sensor/data`.
2. El script oráculo `bridge.py`, que está escuchando activamente en ese tema, recibe el mensaje de inmediato.
3. El script de Python construye, firma y envía una transacción al nodo Anvil, invocando una función en el contrato `SensorData.sol` para registrar los datos.
4. Anvil, configurado para minado instantáneo, procesa la transacción y la incluye en un nuevo bloque sin demora.
5. El script de Python recibe de vuelta el hash de la transacción (Tx Hash) como confirmación de que la operación ha sido registrada exitosamente en la blockchain.
6. Para cerrar el ciclo de retroalimentación, el script publica este Tx Hash en un nuevo tema MQTT: `sensor/confirm`.
7. El ESP32, que está suscrito a `sensor/confirm`, recibe este hash y lo muestra en su monitor serie, confirmando visualmente que sus datos han sido anclados de forma inmutable en la blockchain.

Valor de la Arquitectura

Aunque este montaje utiliza una blockchain local, su diseño es funcionalmente **idéntico a un sistema IoT industrial de producción**. Esto lo convierte en un modelo de un valor incalculable para el prototipado rápido, las pruebas de integración y el desarrollo de soluciones industriales que requieren un alto grado de confianza, auditoría y trazabilidad de los datos.

Con esta visión completa de una integración avanzada, estamos listos para sintetizar las lecciones aprendidas a lo largo de estos ejemplos.



Conclusión: Síntesis de Casos de Uso

A través de tres ejemplos de complejidad progresiva, hemos recorrido un camino que va desde una simple transferencia de fondos hasta una sofisticada integración de IoT con una blockchain. Cada paso ha demostrado cómo la combinación de Docker para el aislamiento y Anvil para la simulación determinista de la EVM nos permite construir entornos de desarrollo y prueba que son robustos, portátiles y, sobre todo, predecibles.

El valor fundamental de estos patrones de diseño se cristaliza en su aplicabilidad a casos de uso críticos en el ciclo de vida del desarrollo de software blockchain. La capacidad de empaquetar y ejecutar flujos complejos con un solo comando es invaluable para:

- **Demos técnicas:** Presentar funcionalidades complejas a clientes o stakeholders sin riesgo de fallos por configuración.
- **Workshops:** Permitir a los participantes centrarse en el aprendizaje del código en lugar de en la configuración del entorno.
- **CI/CD local:** Validar de forma automática y fiable que los nuevos cambios no rompen el ciclo de vida completo de un contrato.
- **Pruebas determinísticas:** Garantizar que las pruebas de integración y de lógica de negocio produzcan siempre los mismos resultados, eliminando la incertidumbre.

En última instancia, el dominio de la creación de entornos automatizados y determinísticos no es solo una buena práctica, sino una habilidad esencial para cualquier equipo o profesional que aspire a construir aplicaciones blockchain fiables, seguras y de alta calidad en un ecosistema cada vez más exigente.

Bibliografía

- Antonopoulos, A. M., & Wood, G. *Mastering Ethereum*.
- Espressif Systems. *ESP-IDF Programming Guide*.
- Foundry Project. *Foundry Book & Anvil Documentation*.
- Merkel, D. (2014). *Docker Containers*. ACM.
- OASIS. *MQTT Specification*.
- OpenZeppelin. *Smart Contract Security Guidelines*.
- ConsenSys Diligence. *Smart Contract Best Practices*.

